D424 – Software Capstone

Task 2

Capstone Proposal Project Name: Trip Itinerary

Student Name: Zoey Hankins

Table of Contents

A. Business Problem 3
The Customer and Business Case 3
Fulfillment 3
B. SDLC Methodology 4
B1. Deliverables 6
B2. Deployment Plan and Outcomes 7
Pre-Deployment 7
B3. Project Timeline 8
C. Environments and Costs 10
Programming Environment 10
Environmental Costs 10
Human Resource Requirements 10
D. Validation and Verification 11
Testing Process 11
Bug Handling 11

A. Business Problem

The Customer and Business Case

TravelBuddy Inc. is a new, up-and-coming company launched in early 2025. TravelBuddy Inc. wants to build its reputation by delivering personalized, memorable travel experiences. As the travel industry becomes increasingly in demand and customer expectations continue to increase, the need for clear, professional, and easily accessible trip information has never been more critical.

In the company's first year, TravelBuddy Inc. grew from an agency of 12 travel agents to a network of over 200 certified travel agents. These "Buddies" specialize in all different types of vacations. TravelBuddy Inc. has also partnered with hotels, tour operators, and local guides to ensure that every trip supports local communities and reduces environmental impact.

TravelBuddy Inc. travel agents provide vacation details through a combination of emails, phone calls, texts, a calendar application, and other informational documents. Even though this approach works, it can pose challenges because customers often receive information in multiple formats, making it difficult for the customer to keep track of essential details.

TravelBuddy's goal is to implement a new "Trip Itinerary" software for their travel agents to use for their clients, as well as make it available to the public and individual travelers who do not want to use a travel agent.

Fulfillment

For individual travelers, a well-prepared itinerary can be tremendously beneficial for trip planning and execution. The mobile "Trip Itinerary" application will offer users a place to centralize, organize, and search trip details, such as the hotel they are staying at, check-in and check-out dates, and an excursion list, allowing them to add notes with precise information on anything the user needs or wants to add, such as transportation and booking times.

Using an itinerary is just as beneficial for TravelBuddy Inc. travel agents as it is for individual travelers. A well-prepared itinerary demonstrates the agent's knowledge of travel and attention to detail, which grants the agent credibility. By delivering seamless travel experiences, clients will be loyal, make repeat bookings, and refer their company to others. It also improves operational efficiency because an itinerary reduces the need for continuous, repetitive communication and last-minute reach-outs. These benefits give the travel agent more time to serve more clients to their liking and grow their reputation.

A travel itinerary is critical to success for both individuals and travel agents. Itinerary creation should be standard for every travel booking at TravelBuddy Inc. because it delivers peace of mind and a richer travel experience for individuals, while growing brand reputation, efficiency, and profitability for travel agents.

B. SDLC Methodology

One of the main goals is to deliver a high-quality product quickly. So, the Agile Scrum methodology will be the software development life cycle for this project. Agile Scrum methodology is a highly collaborative, iterative approach to software development projects that emphasizes flexibility, adaptability, and continuous improvement. Agile encourages breaking projects into smaller increments known as “sprints” and emphasizes delivering valuable content to consumers in short cycles.

Scrum enables the development team to focus on delivering key product features while remaining responsive to TravelBuddy’s feedback. In the Scrum method, daily stand-up meetings review anything that needs to be changed, added, or removed. These meetings foster the communication that ensures the final product aligns with the company’s end goal.

Phase 1 is called “Product Backlog” and involves breaking needed features into user stories, written from the user’s perspective and focused on what the user wants to do; prioritizing them by business value and dependencies; and involving stakeholders in discussions about the final product. This phase is essentially about all members coming together to define the end goal. The main goal of this project is to create a mobile application that has functionality, ease of use, and security for TravelBuddy travel agents and individual travelers.

Phase 2 is called “Sprint Planning”. Sprint planning is usually at the beginning of each Sprint cycle. This phase is simply taking the requirements for all the end goals or “tasks” and splitting them into manageable tasks. Here, the development team will identify the features that need to be added or modified, and plan how to split them across “Sprint 1”, “Sprint 2”, “Sprint 3”, etc. The development team then estimates the effort required, and other team members commit to completing specific sets of items.

Phase 3 is called “Sprint Execution.” During this phase, the development team will execute all the sprints planned in Phase 2. Each sprint lasts 4 to 7 days, depending on the task’s demands. The development team will do one sprint at a time, hold a daily stand-up meeting to review progress and adjust objectives as needed, move on to the next phase, and complete the remaining sprints in the preceding iterations.

Sprint 1 will consist of creating a login/sign-up page for users or a guest option, where the application user does not have to log in. This sprint will produce a functional user login page that supports saved data retrieval upon login (verification logic), account login attempt tracking, and lockouts after too many attempts.

Sprint 2 will consist of creating the vacation entity and vacation interfaces while implementing CRUD (Create, Read, Update, Delete) functionality. This sprint will produce a functional Android application with CRUD functionality, the corresponding user interfaces, and data validation.

Sprint 3 will consist of creating the excursion entity and excursion interfaces, while implementing CRUD functionality again. CRUD will link the vacation and excursion entities in the database for validation logic. This sprint produces a functional Android application with CRUD functionality, a fully implemented excursion entity, the corresponding user interfaces, and data validation.

Sprint 4 will consist of creating a search-and-filter function. This sprint produces a fully functional search-and-filter (validation logic) feature that lets users easily find the trip they are looking for.

Sprint 5 will consist of creating a report generation. This sprint produces a fully functional report of all current vacations, their associated excursions, and hotels, all with dates.

Sprint 6 will be the last. This sprint will focus on any minor changes. No significant changes are here unless absolutely required. These are usually minor cosmetic changes or possibly small logic or validation changes. Major debugging should happen during the sprint for that current task, not here. This sprint will deliver a fully functional and fully developed Android application.

Phase 4, called “Sprint Review and Retrospective,” is also the final phase. During this phase, the development team and stakeholders come together to present all the work completed in the corresponding sprint. This phase is also where the team receives constructive feedback from each other on work ethics and the development process, which the development team will use in future sprints to enhance production and team collaboration.

After all these phases finish, the iteration continues until the team completes all sprints. With every cycle iteration, the team adjusts to improve its dynamics, increase product production and quality, and meet the customer’s needs.

B1. Deliverables

1) Product Backlog
    i) The backlog is a list of new enhancements, features, tasks, bug fixes, or work requirements needed to build the software. This document is information from sources such as market demand, competitor analysis, stakeholder needs, user stories, and business analysis.
2) Wireframe
    i) These are low-fidelity layouts of the mobile application's screens that show the structure and navigation.
3) User Interface Mock-Ups
    i) These are high-fidelity mock-up designs that showcase branding, typography, and colors.
4) Testing and Quality Assurance
    i) These documents contain essential information for stakeholders about testing throughout the process. This document will include a section named "test cases" that describes the test scenarios, the input data, and the expected outcomes. The other section is called "test reports," which summarize the testing that occurred, whether the test passed or failed its expected outcome, and the steps taken to fix the bug in the logic that caused the test to fail.
5) APK
    i) This deliverable will be the fully functional application with a tested and signed APK file.
6) Source Code Repository
    i) This deliverable will be a link to the entire source code, front-end, and back-end.
7) User Guide
    i) This deliverable will include user documentation showing how to perform application actions, along with troubleshooting documentation.
8) Maintenance Guide
    i) This deliverable will be documentation for the company or users that shows deployment setup, architecture, database schema, code structure, how to build the APK, and testing instructions.

B2. Deployment Plan and Outcomes

Pre-Deployment

For pre-deployment, technical documentation will be a deliverable, including UML diagrams and database connectivity guides. The project lead will create user training manuals for TravelBuddy travel agents to help familiarize them with and educate them on the new Trip Itinerary software.

Deployment and Outcomes

For a successful deployment, the production team will publish a static web page to GitHub Pages, with the signed and built Gradle APK attached so it can be downloaded and used on Android devices. On the static deployment page, there will be a technical user guide that goes in depth into the application's features and how to use them, along with a detailed post-development maintenance guide.

The deployed application is called Trip Itinerary, where users can log in to their account to track their trips, including hotel check-in and check-out times, record their excursions, and view all trips in a report. TravelBuddy Inc. will use this application, and it will be free for the public. This is a full-stack MVP Android mobile application developed in Java, using the Room framework as an abstraction layer over the local SQLite database to persist data with a one-to-many relationship, .xml files for design, and using Gradle to build. The system will be deployed by the development team to Git Hub Pages and accessed via a link.

Post-Deployment

Post-deployment testing will include two methods: members of the production team downloading the APK file and testing it in an Android emulator using Android Studio, and downloading the APK to an Android device and testing it there. On the deployment page, there will be a maintenance guide, a user guide, and a troubleshooting document for TravelBuddy to use for post-deployment maintenance.

B3. Project Timeline

Phase	Milestone/Task	Resources	Dependencies	Deliverable	Description	Dates w/ Duration
Research & Planning	Decides requirements and project scope.	Stakeholder, project lead, development team.	Business Case.	Product backlog, user stories, business requirements.	Defining the problem, identifying the users, and the need for this software. Gathering a plan and splitting the team up into their respective groups.	2/12/2026 – 2/16/2026 4 days
UI/UX Design	Create a software design.	UI/UX design team, development team.	Stakeholder design review and approval.	Wireframes, user interface mock-ups, and software design documents.	This phase maps the user experience and any visual and system design.	2/17/2026 – 2/20/2026 3 days
Front-end Development	Building the application's front-end.	Development team (front-end developers).	Stakeholder design review and approval, UI/UX designs finished.	Prototype, mock-up, source code.	Building user interfaces, implementing responsive design, and creating interactive elements that function correctly.	2/21/2026 – 2/28/2026 1 week (7 days)
Back-end Development	Building the application's back-end.	Development team (back-end developers),	Front-end development complete.	Fully functional MVP prototype of the mobile application.	Develop databases, APIs, authentication, server logic, and business functionality.	3/1/2026 – 3/8/2027 1 week (7 days)

Trip Itinerary

Quality & Assurance Testing	Testing the application's features.	QA engineers, developers.	The mobile application should be fully functional and require no further improvements unless something comes to light.	Test plan, test cases, test data, test execution report, and a fully compiled APK file.	Testing to identify and fix bugs, functional security testing, and performance testing.	3/9/2026 – 3/11/2026 2 days
Deployment & Launch	Deploying the application page for download.	DevOps engineer, development team.	QA testing should be complete, modifications made, and the application packaged as an APK.	Fully released GitHub static page with APK link download.	This phase is where the production environment gets set up, migrates data, and then releases the software	3/12/2026 – 3/14/2026 2 days
Maintenance & Updates	Maintenance for the application after release.	Project lead, QA engineer, developers.	The application should be fully launched and deployed.	Maintenance guide, release notes.	Providing security updates, bug fixes, updates of features, and continuously improving the software.	Ongoing

C. Environments and Costs

Programming Environment

Device Hardware: The development team will need a laptop with sufficient speed and storage, running the latest version of Windows.

Database: The database used will be Room (SQLite), which stores all data locally, enabling fast, private data retrieval even when offline.

IDE & Code Editor, Emulator, Compiler, Tester & Debugger: The free version of Android Studio will serve as the code editor for this Java application. As it also includes an Android emulator, debugging tools, a profiler, and the ability to sign and build an APK using Gradle, this is the perfect choice because it offers so many tools in one place.

Version Control: All version control will be free through Git and GitHub. Using Git and GitHub allows the entire development team to collaborate more easily and stay up to date on all code changes and concerns.

Deployment & Hosting: This application will be a downloadable APK file hosted on a free, static GitHub Pages site.

Environmental Costs

Developer Device Hardware (Work Laptops): $1,500- $2,000 per developer, depending on the development team's decision. The team will have 6 developers so that the hardware cost will be $9,000-$12,000.

Human Resource Requirements

The development team consists of 6 developers who work 40 hours per week at $50/hr.

1. Labor Per Developer

40 hours × $50 an hour = **$2,000 per developer per week.**

2. Total Labor Cost

$2,000 a week × 6 Developers = **$12,000 a week in labor.**

All labor costs will be **$12,000 a week.**

D. Validation and Verification

Testing Process

Unit Testing: To verify that all functions, methods, and components work as intended, developers will run unit tests with JUnit. JUnit works by detecting and running methods marked as tests and comparing the actual output of the code with the expected output. If everything matches, the test passes, and if not, JUnit immediately flags the problem. Instead of waiting until the very end of a project to uncover bugs, JUnit helps identify issues as soon as code is written or changed. This early feedback loop reduces time spent debugging and assures developers that their code is behaving as expected. JUnit also works efficiently with GitHub and other automation tools to maintain code quality. Before moving on to integration testing, all code must pass all tests here to prevent minor issues from becoming bigger problems later and to prevent developers from having to work backwards.

Integration Testing: After unit testing, the QA engineers and developers will perform integration testing. Integration testing, using JUnit, involves combining two or more software modules and testing them together. It evaluates whether the modules interact correctly with specified requirements. Integration testing includes testing interfaces, data transfer, and communication protocols that bind the modules together. It goes beyond unit testing, which focuses on verifying the functionality of isolated components such as functions, methods, or classes. While unit testing tells us whether a component works on its own, integration testing verifies that it functions correctly when combined with other elements as intended.

User Acceptance Testing (UAT): Once integration testing is complete, the QA engineers, stakeholders, project lead, and clients perform UAT. User acceptance testing identifies bugs in systems, software, and networks that might cause problems for the end-users. Using UAT validates that the software developed can handle all required tasks and meet the specifications. First, a UAT template is needed to ensure the application meets the business case and simulates a real user experience. There needs to be test scenarios and test cases with test data, along with clear, actionable steps to test. Engineers and developers document any issues encountered during test case execution. Once the bugs get fixed, clients must rerun the tests to verify that there are no more bugs. Finally, ensure the software aligns with the business case and objectives, and that it addresses and fulfills the promises made to the business to meet their needs.

Bug Handling

When a bug gets caught by the testing tool of choice, which in this case is JUnit, these bugs are spotted during test runs and are flagged and logged immediately, with an assigned priority ranging from low to critical. A bug gets logged, and then a developer gets assigned to it for review. Once a developer starts analyzing test results and logs, the developer can patch the code and update the bug with the label “Fixed”. After the bug is labeled “Fixed,” the system automatically retests using unit and integration testing, then returns to UAT to ensure the

developer fixed it correctly. If the developer fixed it, the bug is marked as "Verified”. The bug is then “Closed,” pushed to version control, and the application gets recompiled.

Testing concludes once three distinct events occur. The entire testing plan must be successfully executed by the development and QA teams, as the goal is to validate that the application and software meet all business requirements and function as intended. If all critical tests and user testing pass, further testing is not required. The next event is stakeholder satisfaction. Stakeholders determine whether the product's quality is satisfactory and whether there is sufficient evidence of detailed testing; this validation must occur before the last event. Lastly, if there is a point in the project where no further feedback or iteration is possible, this is the final indicator that stops testing and prepares for release.